

TECHNICAL REFERENCE GUIDE

Four Foreground-Masking Optimisation Objective Functions

SEP and MTOjects | Spike Gate and Toy Objects

Purpose: Record the exact objective functions minimised by the four Optuna pipelines, explain every component, and document why each objective balances foreground recovery against damage to genuine galaxy data.

Code-derived reference | updated 2026-08-16

1. Scope and terminology

The project has four optimiser-to-batch pathways: SEP Spike Gate, MTOBJECTS Spike Gate, SEP Toy Objects, and MTOBJECTS Toy Objects. Optuna minimises an objective during the optimisation stage. The subsequent all-galaxy batch does not evaluate or minimise an objective; it loads the best parameter JSON and applies those parameters to the full sample.

The stability study repeated each optimiser with different reproducible seeds. A seed changes the selected 20-galaxy calibration sample, but the mathematical objective remains the same for that methodology. Therefore 'four batches' refers here to the four engine-method combinations, not four different formulae for four random seeds.

Interpretation rule: All four objectives are minimised. Smaller is better. In the toy-object optimisers, a desirable positive recovery score is negated only after any method-specific feasibility requirements are met; recovery-infeasible candidates receive explicit large positive objectives.

2. At-a-glance comparison

Pipeline	Primary reward	Primary restraint
MTOBJECTS Spike Gate	Cover labelled bar-profile spikes	Avoid masking non-spike profile samples and excessive image area
SEP Spike Gate	High mean and worst-galaxy spike coverage	Caps and penalties for image loss, profile loss, and long bridges
MTOBJECTS Toy Objects	Pixel F-score, per-toy recall, and toy detection rate	Minimum recovery gates, incremental masking, false positives, a 15% hard masking cap, and final cross-fold release criteria
SEP Toy Objects	Pixel recall, F-score, per-toy recall, and toy detection rate	Incremental masking, false positives, and a hard masking cap

3. Shared metric definitions

Bars above symbols denote arithmetic means across the selected galaxy cases. 'Incremental mask' means pixels masked in the injected image but not in the same galaxy's baseline mask; this isolates the response caused by the synthetic toys.

Metric	Operational definition
Spike coverage	Fraction of predefined bar-profile spike samples intersected by the mask; higher is better.
Non-spike profile fraction	Fraction of non-spike bar-profile samples masked; lower means less collateral profile loss.
Profile affected fraction	Fraction of all sampled major-axis positions affected by masking.
Masked fraction	Fraction of all image pixels masked.
Profile change	Median absolute log10 change between the original profile and its bridge-filled counterpart.
Recall	Toy-truth pixels recovered divided by all toy-truth pixels.
Precision	Recovered toy-truth pixels divided by all incremental masked pixels.
F-score	Harmonic mean of recall and precision: $2RP/(R+P)$.
Mean toy recall	Mean recall computed separately for each injected toy.
Toy detection rate	Fraction of toys for which at least 50% of the truth footprint was recovered.
False-positive fraction	Incremental masked pixels outside toy truth, divided by available non-truth pixels.

4. MTOjects Spike Gate objective

The MTOjects Spike Gate optimiser uses a fixed weighted loss:

$$J_{MT, spike} = 12(1 - C_{mean}) + 4N_{mean} + 2M_{mean} + 1.5A_{mean} + 1.0D_{mean}$$

Term	Meaning	Rationale
$12(1 - C_{mean})$	Missed mean spike coverage	Dominant term: genuine spike samples should be removed.

4N_mean	Masked non-spike profile fraction	Discourages deleting valid bar-profile structure.
2M_mean	Whole-image masked fraction	Protects the two-dimensional galaxy image.
1.5A_mean	Profile affected fraction	Penalises broad intervention along the major axis.
1.0D_mean	Median log-profile change	Penalises distortion introduced by bridge filling.

Rationale. The 12:4:2:1.5:1 weighting makes spike removal the principal goal, while still charging for collateral masking at both image and profile levels. It is a soft trade-off: unlike the other three optimisers, this implementation has no explicit maximum-masking cap in its aggregate objective.

5. SEP Spike Gate objective

The stability runs invoked the optimiser defaults, giving the following base objective:

$$J_{\text{base}} = 24(1-C_{\text{mean}}) + 12(1-C_{\text{min}}) + 2N_{\text{mean}} + 4M_{\text{mean}} + 2A_{\text{mean}} + 0.5D_{\text{mean}}$$

The general implementation also contains a configurable bridge-span term, $B_{\text{weight}} \times B_{\text{mean}}$. Its default weight in the stability optimiser is zero, so it did not contribute unless a later automation recipe overrode it.

$$J_{\text{SEP,spike}} = J_{\text{base}} + [10 + 100E_{\text{M}} + 80E_{\text{A}} + 80E_{\text{N}} + 4E_{\text{B}}] \quad \text{if any cap is exceeded}$$

Here each E is positive excess above its configured cap: maximum image-masked fraction, mean profile-affected fraction, mean non-spike fraction, or maximum bridge span. Under the stability defaults, the active image cap was 0.15; the profile caps were 1.0 and the bridge cap was effectively unlimited, so the latter constraints normally added no penalty.

Design choice	Rationale
Mean coverage weight 24	Rewards strong performance across the sample.
Minimum coverage weight 12	Prevents a good average from hiding a badly served spike galaxy.
Fixed 10-point cap jump	Separates cap-breaking candidates sharply from feasible candidates.
Scaled cap excess	Makes increasingly severe data loss progressively less competitive.

6. MTOjects Toy Objects objective

Only incremental masking beyond the unaltered baseline is scored. Let I be the total number of incremental masked pixels, Q the toy detection rate, T_mean the mean per-toy recall, M_mean the mean incremental masked fraction, and $\max(M)$ the worst-galaxy incremental masked fraction.

Recovery reward: $R_MT = 0.45F_mean + 0.35T_mean + 0.20Q$

Data-loss term: $L_MT = 0.50M_mean + 0.10 \min(P_false, 1)$; net score $S_MT = R_MT - L_MT$.

Recovery feasibility gate: a trial is infeasible when $I = 0$, $Q < 0.25$, or $T_mean < 0.20$. Its minimisation objective is $J_infeasible = 50 + 20 \max(0, 0.25 - Q) + 20 \max(0, 0.20 - T_mean) + 50 \mathbb{1}[I=0]$. Thus a completely empty incremental mask receives an objective of at least 100 rather than zero.

For a recovery-feasible trial, $J_MT, toy = -S_MT$ when $\max(M) \leq 0.15$. If the masking cap is exceeded, $J_MT, toy = 10 + 100[\max(M) - 0.15] + L_MT - R_MT$.

Rationale. The feasibility gate prevents the optimiser from preferring a zero-mask solution merely because it has no masking or false-positive penalty. Among trials that recover a minimum amount of toy signal, the weighted score then favours pixel F-score, fairness across individual toys, and toy detection while restraining collateral masking. The 15% worst-image cap continues to reject excessive data loss.

Final cross-validation release gate. The objective above governs individual Optuna trials. A fold winner is released to the 182-galaxy batch only if its common 40-galaxy evaluation has $Q \geq 0.50$, $T_mean \geq 0.30$, $\max(M) \leq 0.15$, and non-zero toy detection and non-zero mean toy recall in every held-out fold. If no candidate satisfies these conditions, the run writes a rejection report and does not start the batch.

7. SEP Toy Objects objective

SEP uses the same baseline-subtracted incremental-mask framework, but places an explicit additional reward on aggregate pixel recall:

$R_SEP = 0.45R_mean + 0.20F_mean + 0.25T_mean + 0.20Q$

$L_SEP = 0.35M_mean + 0.05 \min(P_false, 1)$

$S_SEP = R_SEP - L_SEP$; $J_SEP, toy = -S_SEP$ when $\max(M) \leq 0.15$

$J_SEP, toy = 10 + 100[\max(M) - 0.15] + L_SEP - R_SEP$ when the cap is exceeded

Rationale. SEP's reward emphasises recovering toy pixels, with supporting terms for precision-balanced F-score, fairness across individual toys, and successful whole-toy detection. The recovery weights sum to 1.10; this is exactly what the implemented code uses and is not normalised. SEP's data-loss penalties

remain lighter than the MTObjects recovery follow-up (0.35 and 0.05 rather than 0.50 and 0.10), allowing a somewhat more aggressive search before the shared 15% hard cap is breached.

8. Why baseline subtraction matters for toy scoring

Both toy-object optimisers first process the unaltered galaxy and retain its baseline mask. For a trial, they process the injected image and score only:

```
incremental_mask = injected_image_mask AND NOT baseline_mask
```

This prevents real foreground sources that were already masked before injection from being counted as toy-related false positives. It also means the objective measures sensitivity to the synthetic intervention rather than rewarding an algorithm merely for being globally aggressive.

9. Practical interpretation

Observation	Meaning
Lower objective	Better according to the encoded recovery-versus-damage trade-off.
Negative toy objective	A positive net recovery score from a recovery-feasible toy trial with no cap violation; more negative is better.
Toy objective above 10	For a recovery-feasible toy trial, the maximum incremental masked-fraction cap was exceeded.
Spike objective near zero	High spike coverage with little collateral masking or profile distortion.
Different best parameters by seed	Expected sampling variability; not a change in the objective formula.
MTOBJECTS Toy objective ≥ 50	The trial failed at least one recovery feasibility condition; an empty incremental mask receives at least 100.
No MTOBJECTS batch output	No fold candidate passed the final common-40 and per-fold release gates; rejection is intentional.
Caution: Objective values are only directly comparable within the same optimiser definition and weight configuration. Do not compare the numeric objective from SEP Toy Objects with MTOBJECTS Spike Gate as if they share a common scale.	

10. Implementation basis

This reference was derived directly from the `aggregate_score` and `score_case` implementations in:

Source: `Foreground Masking\Optimisation\optimise_spike_gate_MTOBJECTS.py`

Source: `Foreground Masking\Optimisation\optimise_spike_gate_SEP.py`

Source: `Foreground Masking\Optimisation\optimise_toy_objects_MTOBJECTS.py`

Source: `Foreground Masking\Optimisation\optimise_toy_objects_SEP.py`

Source: `Foreground Masking\Automation\run_optimiser_stability_study.ps1`

Later specialised automation recipes changed configurable weights and caps. The MTOBJECTS Toy Objects recovery follow-up additionally introduced the explicit recovery feasibility and final cross-validation release gates documented in Section 6. Other variants use their recorded configuration and should not be assumed to share identical objective scales.